

Name : Narendra Kumar S

Date : 19-08-2025

Task:

Create additional table called users. put api post request to insert user into it in fast api. Additionally, when request to list music api is called, only a valid user is allowed to get the response else raise HTTPException with error code

Solution:

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import sessionmaker, Session, declarative_base
from fastapi import FastAPI, Depends, Request, HTTPException
from typing import List
from pydantic import BaseModel
import uvicorn

SQLALCHEMY_DATABASE_URL = "sqlite:///C:\\Users\\narendrakumar.s\\songs.db"
engine = create_engine(SQLALCHEMY_DATABASE_URL,
connect_args={"check_same_thread": False})

SessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)
Base = declarative_base()

class SongModel(Base):
    __tablename__ = "songs"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    playtime = Column(Integer, nullable=False)
    album = Column(String(100), nullable=False)

class UserModel(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False, unique=True)
    age = Column(Integer, nullable=False)
```

```

Base.metadata.create_all(bind=engine)

app = FastAPI()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

class Song(BaseModel):
    id: int
    name: str
    playtime: int
    album: str
    class Config:
        orm_mode = True

class SongDTO(BaseModel):
    name: str
    class Config:
        orm_mode = True

class User(BaseModel):
    id: int
    name: str
    age: int
    class Config:
        orm_mode = True

@app.post("/add_song/", response_model=Song)
def add_song(user_name: str, song: Song, db: Session = Depends(get_db)):

    user = db.query(UserModel).filter(UserModel.name == user_name).first()
    if user is None:

```

```

        raise HTTPException(status_code=401, detail="No user found -
Unauthorized access")

    new_song = SongModel(name=song.name, playtime=song.playtime,
album=song.album)
    db.add(new_song)
    db.commit()
    db.refresh(new_song)
    return new_song

@app.get("/list_songs", response_model=List[Song])
def list_songs(db: Session = Depends(get_db)):
    return db.query(SongModel).all()

@app.get("/get_song/{song_id}", response_model=SongDTO)
async def get_song(song_id: int, db: Session = Depends(get_db)):
    song = db.query(SongModel).filter_by(id=song_id).first()
    if not song:
        raise HTTPException(status_code=404, detail="Song not found")
    return SongDTO(name=song.name)

@app.post("/add_user", response_model=User)
def add_user(user: User, db: Session = Depends(get_db)):
    new_user = UserModel(name=user.name, age=user.age)
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
    return new_user

@app.get("/list_users", response_model=List[User])
def list_users(db: Session = Depends(get_db)):
    return db.query(UserModel).all()

if __name__ == "__main__":
    uvicorn.run("Songs:app", host="127.0.0.1", port=8000, reload=True)

```

Output:

FastAPI 0.1.0 OAS 3.1
/openapi.json

default

- POST /add_song/ Add Song
- GET /list_songs List Songs
- GET /get_song/{song_id} Get Song
- POST /add_user Add User
- GET /list_users List Users

Schemas

HTTPValidationError > Expand all object

127.0.0.1:8080/docs/#/default/add_user_add_user_post

GET /get_song/{song_id} Get Song

POST /add_user Add User

Parameters

No parameters

Request body **required** application/json

Edit Value | Schema

```
{
  "id": 0,
  "name": "Santham",
  "age": 45
}
```

127.0.0.1:8080/docs#/default/add_user_add_user_post

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/add_user' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 0,
    "name": "Santham",
    "age": 45
  }'
```

Request URL

http://127.0.0.1:8080/add_user

Server response

Code	Details
200	Response body <pre>{ "id": 5, "name": "Santham", "age": 45 }</pre> Response headers <pre>content-length: 34 content-type: application/json date: Tue, 19 Aug 2025 12:11:10 GMT server: uvicorn</pre>

127.0.0.1:8080/docs#/default/list_users_list_users_get

`-H 'accept: application/json'`

Request URL

http://127.0.0.1:8080/list_users

Server response

Code	Details
200	Response body <pre>[{ "id": 1, "name": "Ajith", "age": 55 }, { "id": 2, "name": "Kamal", "age": 67 }, { "id": 3, "name": "Vijay", "age": 55 }, { "id": 4, "name": "Rajini", "age": 75 }, { "id": 5, "name": "Santham", "age": 45 }]</pre> Response headers

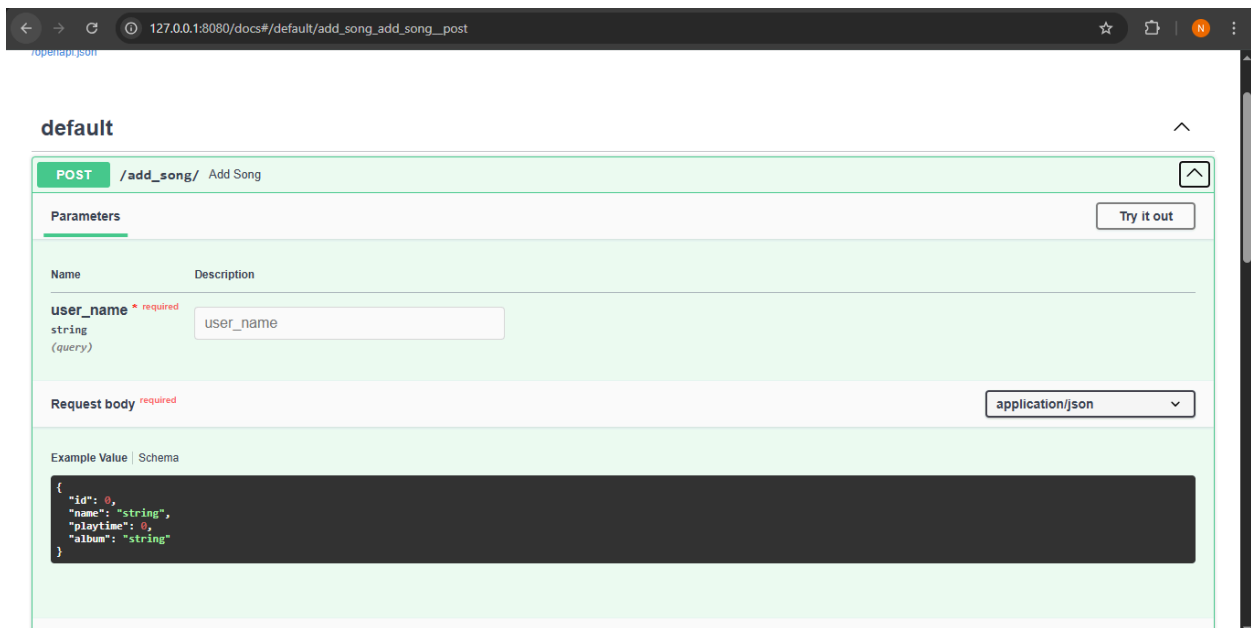
Only allow adding a song if the user exists in the User table Funtion

```
@app.post("/add_song/", response_model=Song)
def add_song(user_name: str, song: Song, db: Session = Depends(get_db)):

    user = db.query(UserModel).filter(UserModel.name == user_name).first()
    if user is None:
        raise HTTPException(status_code=401, detail="No user found -
Unauthorized access")

    new_song = SongModel(name=song.name, playtime=song.playtime,
album=song.album)
    db.add(new_song)
    db.commit()
    db.refresh(new_song)
    return new_song
```

Output:



The screenshot displays a REST client interface for a POST endpoint `/add_song/`. The interface is organized into several sections:

- Method and Path:** A green bar at the top indicates the `POST` method and the path `/add_song/`, with a description "Add Song".
- Parameters:** A section below the header showing query parameters. It includes a table with columns "Name" and "Description". A single parameter, `user_name`, is listed as a required string query parameter. A "Try it out" button is located to the right.
- Request body:** A section indicating that a request body is required. A dropdown menu shows the content type is set to `application/json`.
- Example Value:** A section at the bottom showing an example JSON object:

```
{
  "id": 0,
  "name": "string",
  "playtime": 0,
  "album": "string"
}
```

127.0.0.1:8080/docs#/default/add_song_add_song__post

POST /add_song/ Add Song

Parameters

Cancel Reset

Name Description

user_name * required
string
(query)

Ajith

Request body required

application/json

Edit Value Schema

```
{
  "id": 0,
  "name": "Vikram",
  "playtime": 133,
  "album": "vikram"
}
```

127.0.0.1:8080/docs#/default/add_song_add_song__post

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/add_song/?user_name=Ajith' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 0,
    "name": "Vikram",
    "playtime": 133,
    "album": "vikram"
  }'
```

Request URL

http://127.0.0.1:8080/add_song/?user_name=Ajith

Server response

Code Details

200

Response body

```
{
  "id": 0,
  "name": "Vikram",
  "playtime": 133,
  "album": "vikram"
}
```

Response headers

```
content-length: 56
content-type: application/json
date: Tue, 19 Aug 2025 12:15:02 GMT
server: uvicorn
```

Responses

Example for user not exist in the table

127.0.0.1:8080/docs#/default/add_song_add_song_post

POST /add_song/ Add Song

Parameters

Cancel Reset

Name	Description
user_name * required	
string (query)	Arnold

Request body required

application/json

Edit Value | Schema

```
{
  "id": 0,
  "name": "Vikram",
  "playtime": 133,
  "album": "vikram"
}
```

127.0.0.1:8080/docs#/default/add_song_add_song_post

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/add_song/?user_name=Arnold' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 0,
    "name": "Vikram",
    "playtime": 133,
    "album": "vikram"
  }'
```

Request URL

http://127.0.0.1:8080/add_song/?user_name=Arnold

Server response

Code	Details
401 Undocumented	Error: Unauthorized

Response body

```
{
  "detail": "No user found - Unauthorized access"
}
```

Response headers

```
content-length: 48
content-type: application/json
date: Tue, 19 Aug 2025 12:16:00 GMT
server: uvicorn
```

Responses

Code	Description	Links
------	-------------	-------

Database Screenshots

User table

The screenshot shows the DB Browser for SQLite interface. The main window displays the 'users' table with the following data:

id	name	age
1	Ajith	55
2	Kamal	67
3	Vijay	55
4	Rajini	75
5	Santham	45

The right sidebar shows the 'Edit Database Cell' dialog with 'Mode: Text' and 'Type: NULL; Size: 0 bytes'. The bottom status bar indicates 'UTF-8' encoding.

Song Table

The screenshot shows the DB Browser for SQLite interface. The main window displays the 'songs' table with the following data:

id	name	playtime	album
5	Kanima	167	Retro
6	Vathi coiming	230	Master
7	Ratha mara	169	Jailer
8	LEO	39	Leo
9	Vikram	133	vikram

The right sidebar shows the 'Edit Database Cell' dialog with 'Mode: Text' and 'Type: Text / Numeric; Size: 1 character(s)'. The bottom status bar indicates 'UTF-8' encoding.

